Сборки и метаданные

Сборки и метаданные

СБОРКА И МОДУЛЬ (ASSEMBLY & MODULE)

Сборка и модуль (Assembly & Module)

- В .NET framework Assembly - это минимальная единица развертывания, представляющая собой файл EXE или DLL.
- Сборка может содержать один или несколько файлов, они могут включать такие вещи, как файлы ресурсов или модули (встроенные).

Сборка и модуль (Assembly & Module)

- Модули – это файлы кода в сборке.
- Модуль - это единица компиляции.
- Модуль содержит метаданные типа.
- Модуль не может быть развернут отдельно, он должен быть связан в сборку

Сборка и модуль (Assembly & Module)

Топологию сборки можно просмотреть с помощью **дизассемблера IL (ildasm.exe)** .

Сборки (.Net Framework)

Скомпилированный проект представляет собой файл с расширением

.exe (исполняемый файл) или

.dll (библиотека классов)

Сборки (.Net 5.0)

Публикация приложения как зависимого от платформы создает межплатформенный двоичный файл в виде файла ***.dll*** и исполняемый файл (***.exe***) для конкретной платформы , предназначенный для текущей платформы.

Сборки

Сборки бывают:

- *локальные* (или *сборки со слабыми именами*)
- *глобальные* (или *сборки с сильными именами*)

Сборки

Локальная сборка при выполнении приложения должна находиться в том же каталоге, что и приложение, которое использует эту сборку.

Локальные сборки обеспечивают простоту развёртывания приложения (все его компоненты сосредоточены в одном месте) и изолированность компонентов.

Имя локальной сборки – *слабое имя* – это имя файла сборки без расширения.

Сборки

Для глобальных сборок в .NET используется специальное защищённое
хранилище сборок (*Global Assembly Cache*, *GAC*).

Сборки

Начиная с .NET Framework 4 расположение глобального кэша сборок по умолчанию — **%windir%\Microsoft.NET\assembly**.

Сборки

Сборки, развернутые в глобальном кэше сборок, должны иметь строгие имена. При добавлении сборки в глобальный кэш сборок выполняется проверка целостности всех входящих в нее файлов. Кэш выполняет такую проверку целостности, чтобы гарантировать, что сборка не была изменена (например, если файл был изменен, но изменения не были отражены в манифесте сборки).

Сборки

Установить сборку в глобальный кэш сборок можно с одним из следующих компонентов:

- Установщик Windows
- Средство глобального кэша сборок

Сборки

Пример использования средства глобального кэша сборок:

gacutil -i <assembly name>

Пример использования средства глобального кэша сборок из VisualStudio – см

https://docs.microsoft.com/ru-ru/troubleshoot/dotnet/csharp/install-assembly-gac

В .NET Core сборки со строгими именами **не** предоставляют материальных преимуществ.

Сборки и метаданные

СТРУКТУРА СБОРКИ

Структура сборки

Сборка может состоять из четырех элементов:

- Манифест сборки - содержит метаданные сборки.
- Метаданные типов.
- Код промежуточного языка Microsoft (MSIL), реализующий типы. Он генерируется компилятором из одного или нескольких файлов исходного кода.
- Набор ресурсов .

Обязателен только манифест сборки, но необходимы либо типы, либо ресурсы, чтобы дать сборке какие-либо значимые функции.

Структура сборки



https://docs.microsoft.com/en-us/dotnet/standard/assembly/contents

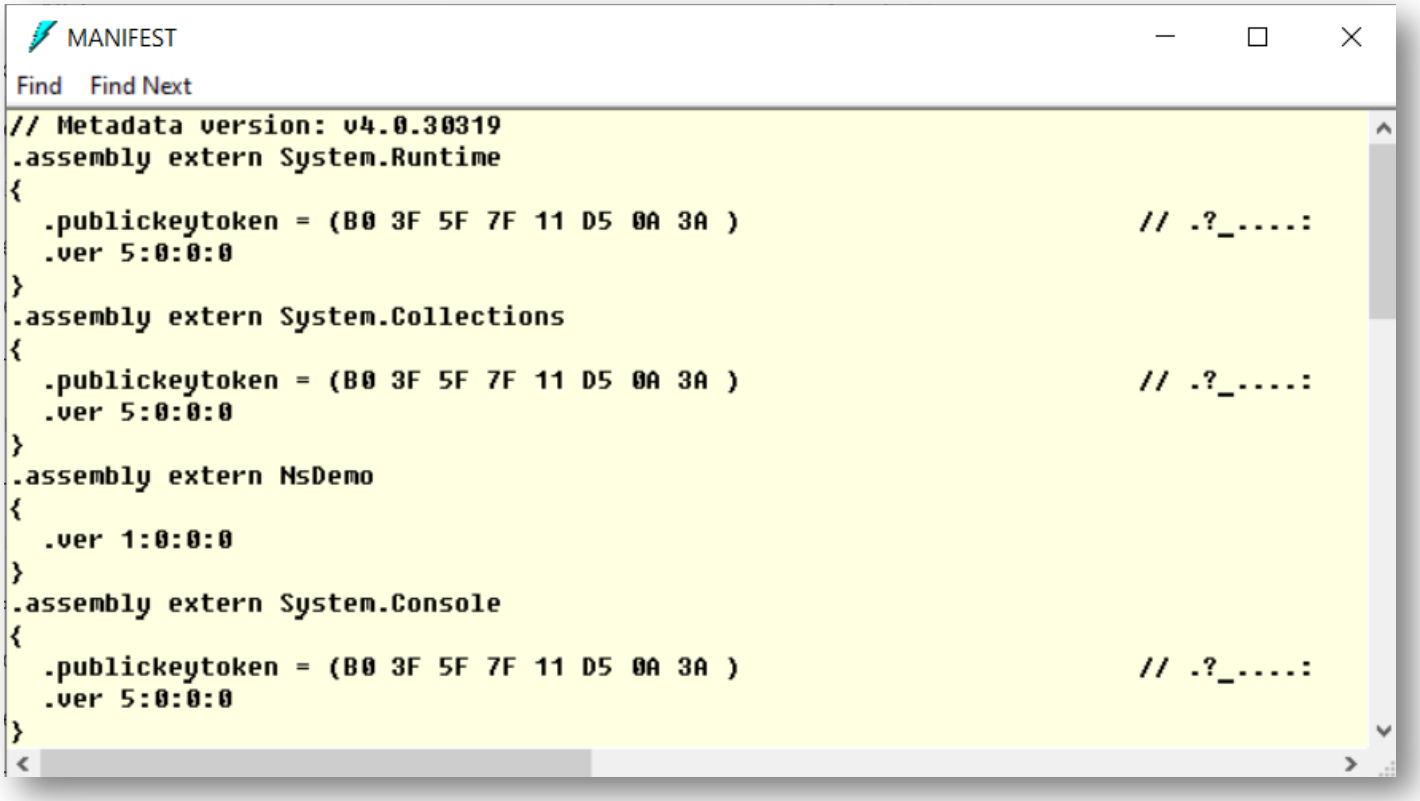
Манифест сборки

Каждая сборка, статическая или динамическая, содержит набор данных, описывающих, как элементы в сборке соотносятся друг с другом. Манифест сборки содержит эти метаданные сборки. Манифест сборки содержит все метаданные, необходимые для указания требований к версии сборки и удостоверения безопасности, а также все метаданные, необходимые для определения области сборки и разрешения ссылок на ресурсы и классы.

Манифест сборки

- Перечисляет файлы, составляющие сборку.
- Управляет тем, как ссылки на типы и ресурсы сборки сопоставляются с файлами, содержащими их объявления и реализации.
- Перечисляет другие сборки, от которых зависит сборка.
- Обеспечивает уровень косвенного обращения между потребителями сборки и деталями реализации сборки.
- Отображает сборку с самоописанием.

Манифест сборки



Сборки и метаданные

МЕТАДАННЫЕ ТИПА

Метаданные типа

Метаданные являются описанием всех типов в сборке и их элементов

Метаданные типа

Рефлексия (reflection, отражение) - это процесс обнаружения типа во время выполнения.

Метаданные типа

Используя службы рефлексии можно программно получать информацию метаданных с помощью дружественной объектной модели.

Метаданные типа

Например, с помощью рефлексии можно получить:

- список всех типов, содержащихся в данной сборке * .dll или * .exe, включая методы, поля, свойства и события, определенные данным типом.
- динамически обнаруживать набор интерфейсов, поддерживаемых данным типом, параметры метода и другие связанные детали (базовые классы, информация о пространстве имен, данные манифеста и т.д.).

Метаданные типа

Основной функционал рефлексии сосредоточен в пространстве имен **System.Reflection**.

Классы пространства имен System.Reflection

Assembly: класс, представляющий сборку и позволяющий манипулировать этой сборкой

AssemblyName: класс, хранящий информацию о сборке

MemberInfo: базовый абстрактный класс, определяющий общий функционал для классов EventInfo, FieldInfo, MethodInfo и PropertyInfo

EventInfo: класс, хранящий информацию о событии

Классы пространства имен System.Reflection

FieldInfo: хранит информацию об определенном поле типа
MethodInfo: хранит информацию об определенном методе
PropertyInfo: хранит информацию о свойстве
ConstructorInfo: класс, представляющий конструктор
Module: класс, позволяющий получить доступ к определенному модулю внутри сборки
ParameterInfo: класс, хранящий информацию о параметре метода

Класс System.Type

Класс System.Type

Класс System.Type определяет набор членов, которые можно использовать для проверки метаданных типа, большое количество которых возвращает типы из пространства имен System.Reflection.

Класс System.Type

- Метод **FindMembers()** возвращает массив объектов MemberInfo данного типа
- Метод **GetConstructors()** возвращает все конструкторы данного типа в виде набора объектов ConstructorInfo
- Метод **GetEvents()** возвращает все события данного типа в виде массива объектов EventInfo
- Метод **GetFields()** возвращает все поля данного типа в виде массива объектов FieldInfo
- Метод **GetInterfaces()** получает все реализуемые данным типом интерфейсы в виде массива объектов Type

Класс System.Type

- Метод **GetMembers()** возвращает все члены типа в виде массива объектов MemberInfo
- Метод **GetMethods()** получает все методы типа в виде массива объектов MethodInfo
- Метод **GetProperties()** получает все свойства в виде массива объектов PropertyInfo
- Свойство **Name** возвращает имя типа
- Свойство **Assembly** возвращает название сборки, где определен тип

Класс System.Type

- Свойство **Assembly** возвращает название сборки, где определен тип
- Свойство **Namespace** возвращает название пространства имен, где определен тип
- Свойство **IsArray** возвращает true, если тип является массивом
- Свойство **IsClass** возвращает true, если тип представляет класс
- Свойство **IsEnum** возвращает true, если тип является перечислением
- Свойство **IsInterface** возвращает true, если тип представляет интерфейс

Класс System.Type (получение типа)

Получить тип можно тремя способами:

- с помощью ключевого слова **typeof**,
- с помощью метода **GetType()** класса Object
- с помощью статического метода **Type.GetType()**.

Класс System.Type (получение типа)

```
public class Book
    {
        public string BookName { get; set; }
        public Author Author { get; set; }
        public int Pages { get; set; }

        public Book()
        {

        }
        public Book(string name, Author author, int pages)
        {
            BookName = name;
            Author = author;
            Pages = pages;
        }

        public int PagesToRead(int currentPage)
        {
            return Pages - currentPage;
        }
        public override string ToString()
        {
            return $"Название: {BookName}, автор: {Author.Name}, {Pages} страниц";
        }
    }
```

Класс System.Type (получение типа)

```
var type1 = typeof(Book);

var type2 = Type.GetType(nameof(Book));

var book = new Book();
var type3 = book.GetType();
```

Класс System.Type (информация о членах класса)

```
  var myType = Type.GetType("ConsoleApp6.Book",
false, true);

foreach (MemberInfo mi in myType.GetMembers())
{
    Console.WriteLine($"{mi.DeclaringType}
                       {mi.MemberType} {mi.Name}");
}
Console.ReadLine();
```

Класс System.Type (информация о членах класса)

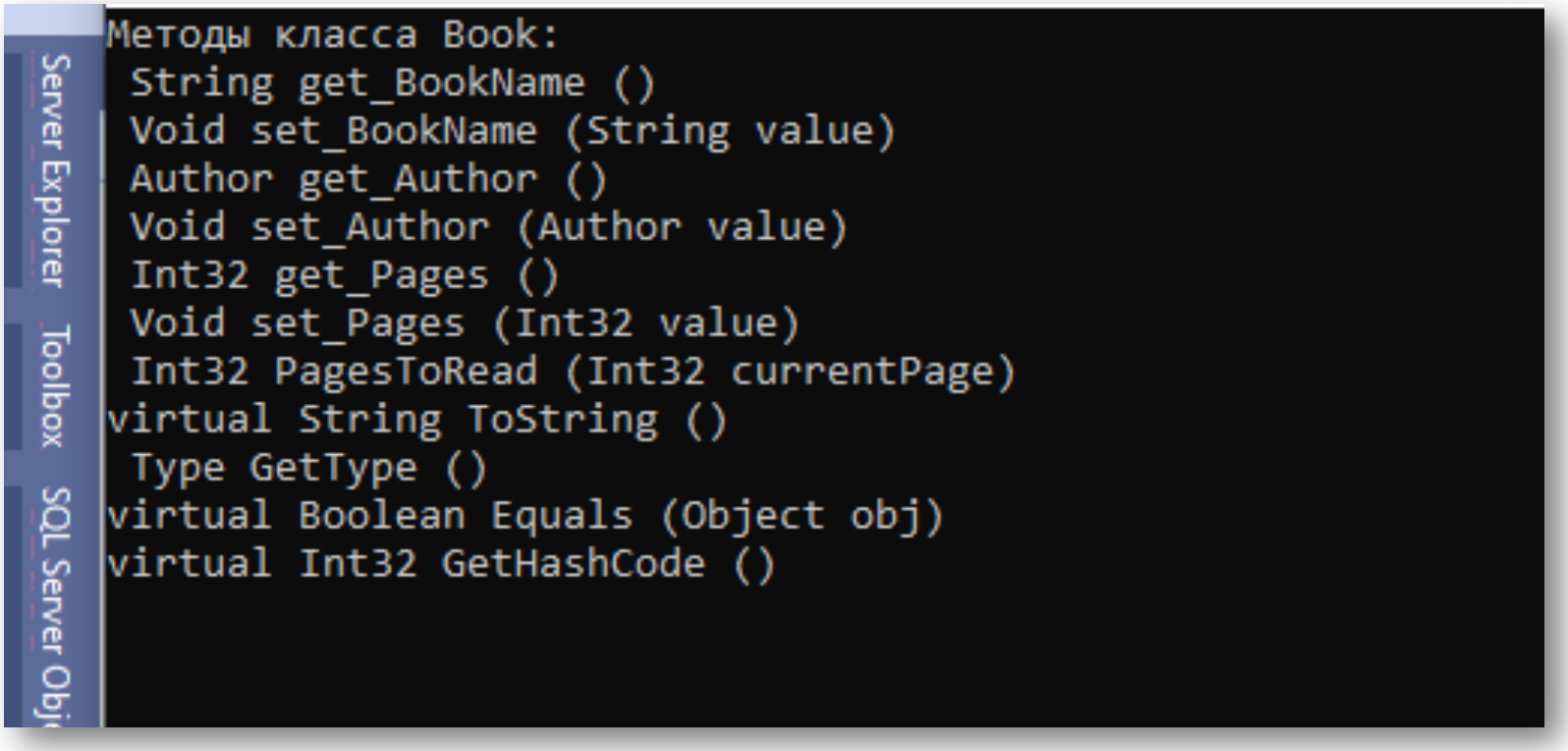
```
ConsoleApp6.Book Method get_BookName
ConsoleApp6.Book Method set_BookName
ConsoleApp6.Book Method get_Author
ConsoleApp6.Book Method set_Author
ConsoleApp6.Book Method get_Pages
ConsoleApp6.Book Method set_Pages
ConsoleApp6.Book Method PagesToRead
ConsoleApp6.Book Method ToString
System.Object Method GetType
System.Object Method Equals
System.Object Method GetHashCode
ConsoleApp6.Book Constructor .ctor
ConsoleApp6.Book Constructor .ctor
ConsoleApp6.Book Property BookName
ConsoleApp6.Book Property Author
ConsoleApp6.Book Property Pages
```

Класс System.Type (информация о методах класса)

```
Console.WriteLine($"Методы класса {myType.Name}:") ;
foreach (MethodInfo method in myType.GetMethods())
{
     var sb = new StringBuilder();

     if (method.IsStatic)
                    sb.Append("static ");
     if (method.IsVirtual)
                    sb.Append("virtual");
     sb.Append($" {method.ReturnType.Name} {method.Name} (");
                //получаем все параметры
     ParameterInfo[] parameters = method.GetParameters();
     for (int i = 0; i < parameters.Length; i++)
     {
         sb.Append($"{parameters[i].ParameterType.Name} {parameters[i].Name}");
         if (i + 1 < parameters.Length) sb.Append(", ");
     }
     sb.Append(")");
     Console.WriteLine(sb);
}
Console.ReadLine();
```

Класс System.Type (информация о методах класса)



Класс System.Type (информация о конструкторах)

```
var myType = Type.GetType("ConsoleApp6.Book", false, true);

Console.WriteLine($"Конструкторы класса {myType.Name}:");
foreach (ConstructorInfo ctor in myType.GetConstructors())
{
     Console.Write(myType.Name + " (");
     // получаем параметры конструктора
     ParameterInfo[] parameters = ctor.GetParameters();
     for (int i = 0; i < parameters.Length; i++)
     {
        Console.Write(parameters[i].ParameterType.Name + " " +
parameters[i].Name);
        if (i + 1 < parameters.Length)
                        Console.Write(", ");
      }
      Console.WriteLine(")");
}
```

Класс System.Type (информация о конструкторах)

```
Конструкторы класса Book:
Book ()
Book (String name, Author author, Int32 pages)
```

Сборки и метаданные

ДИНАМИЧЕСКАЯ ЗАГРУЗКА СБОРОК И ПОЗДНЕЕ СВЯЗЫВАНИЕ

Позднее связывание

Поздним связыванием (late binding) - это механизм, позволяющий создавать экземпляр определенного типа и вызывать его члены **во время выполнения**, а не на этапе компиляции.

Позднее связывание

Пример: переопределенные виртуальные методы

Динамическая загрузка сборок

```
using System;
using System.Collections.Generic;
using System.IO;
using ConsoleApp13;
using System.Configuration;
using Microsoft.Extensions.Configuration;
using System.Xml.Serialization;
```

Динамическая загрузка сборок

При обращении к функционалу этих сборок они автоматически подгружаются.

Динамическая загрузка сборок

Класс **Assembly,** из пространства имен **System.Reflection** позволяет загружать сборку, на которую нет ссылки в проекте.

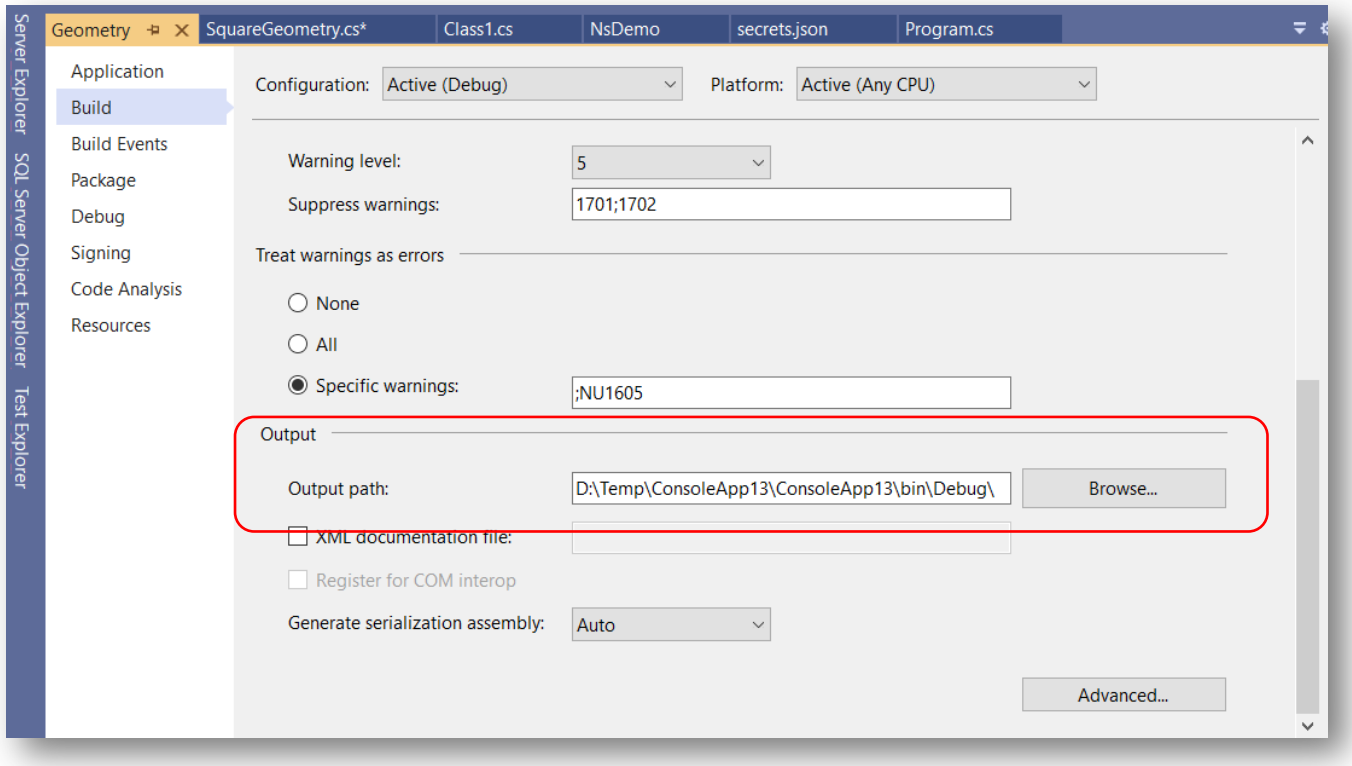
Динамическая загрузка сборок

С помощью динамической загрузки можно реализовать технологию позднего связывания, т.е. создавать экземпляры некоторого типа, а также использовать его во время выполнения приложения.

Динамическая загрузка сборок

```
namespace Geometry
{
    public class SquareGeometry
    {
        public double Height { get; set; }
        public double Length { get; set; }
        public SquareGeometry():this(0,0)
        {          }
        public SquareGeometry(double height, double length)
        => (Height, Length) = (height, length);
        public double GetSquare() => Height * Length;
    }
}
```

Динамическая загрузка сборок



Динамическая загрузка сборок

```
Assembly asm = Assembly.LoadFrom("Geometry.dll");
Type type = asm.GetType("Geometry.SquareGeometry");

var geom1 = asm.CreateInstance("Geometry.SquareGeometry");
var geom2 = Activator.CreateInstance(type,
                                     new object[] { 10, 15 });

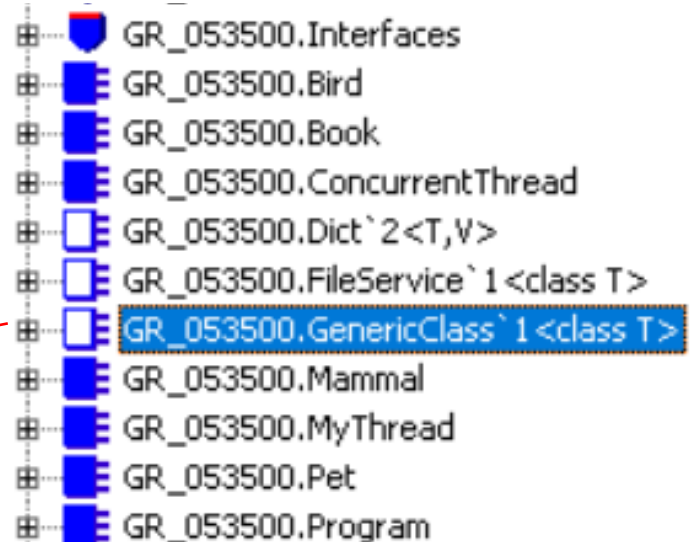
var meth = asm.GetType("Geometry.SquareGeometry")
              .GetMethod("GetSquare");

var result = meth.Invoke(geom2, null);
```

Динамическая загрузка сборок

Создание объекта обобщенного класса:

```
class GenericClass<T> where T : class
    {
    }
```



GenericClass`1

Создание объекта обобщенного класса

```
Type type =
    asm.GetType("Geometry.GenericClass`1");
                .MakeGenericType(typeof(Book));
```

Сборки и метаданные

СБОРКИ НЕУПРАВЛЯЕМОГО КОДА

Сборки неуправляемого кода

Процесс создания моста между управляемым кодом (**managed code**) и неуправляемым кодом (**unmanaged code)** называется Маршалингом (**marshalling**)

Сборки неуправляемого кода

Для вызова функций, экспортированных из неуправляемой библиотеки, приложению .NET Framework требуется прототип функции в управляемом коде, представляющий неуправляемую функцию.

Сборки неуправляемого кода

Чтобы создать прототип, который допускает вызов неуправляемого кода для правильного маршалинга данных, необходимо выполнить указанные ниже действия:

- Примените атрибут DllImportAttribute к статической функции или методу в управляемом коде.
- Замените неуправляемые типы данных на управляемые.

Сборки неуправляемого кода

Описание	тип Windows	ключевое слово C/C++	managed-тип	ключевое слово C#
8-разрядное целое со знаком	CHAR	char	System.SByte	sbyte
8-разрядное целое без знака	BYTE	unsigned char	System.Byte	byte
16-разрядное целое со знаком	SHORT	short	System.Int16	short
16-разрядное целое без знака	WORD и USHORT	unsigned short	System.UInt16	ushort
32-разрядное целое со знаком	INT, INT32, LONG и LONG32	int, long	System.Int32	int
32-разрядное целое без знака	DWORD, DWORD32, UINT и UINT32	unsigned int, unsigned long	System.UInt32	uint
64-разрядное целое со знаком	INT64, LONGLONG и LONG64	__int64, long long	System.Int64	long
64-разрядное целое без знака	DWORDLONG, DWORD64, ULONGLONG и UINT64	unsigned __int64, unsigned long long	System.UInt64	ulong
Число с плавающей запятой	FLOAT	float	System.Double	double

https://docs.microsoft.com/ru-ru/dotnet/framework/interop/marshaling-data-with-platform-invoke

Сборки неуправляемого кода

Пример:

определить и вызвать функцию **MessageBox** в User32.dll, передав в качестве аргумента простую строку.

Сборки неуправляемого кода

```
public class Win32
{
    [DllImport("user32.dll", CharSet = CharSet.Auto)]
    public static extern IntPtr MessageBox(int hWnd,
                String text, String caption, uint type);
}
```

https://docs.microsoft.com/ru-ru/dotnet/framework/interop/platform-invoke-examples

Сборки неуправляемого кода

```
Win32.MessageBox(0, "Hello World",
                    "Platform Invoke Sample", 0);
```

https://docs.microsoft.com/ru-ru/dotnet/framework/interop/platform-invoke-examples

Сборки неуправляемого кода

Подробно – см.

Взаимодействие с неуправляемым кодом [электронный ресурс] - режим доступа: https://docs.microsoft.com/ru-ru/dotnet/framework/interop/

Сборки и метаданные

ДИНАМИЧЕСКИЙ ТИП

Динамический тип

C#, начиная с версии 4.0, вводит новый тип, **dynamic**. Тип в C# является статическим типом, но объект типа dynamic обходит проверку статического типа. В большинстве случаев он работает так, как будто имеет тип object.

Динамический тип

Среда выполнения динамического языка (**DLR**) - это API, представленный в .NET Framework 4. Он предоставляет инфраструктуру, поддерживающую dynamic тип в C #, а также реализацию языков динамического программирования, таких как IronPython и IronRuby.

Динамический тип

```
dynamic item;
item = "1";
item = 2;
```

Динамический тип

```
enum DateFormats
{
    ShortDateString,
    ShortTimeString,
    Ticks
}
```

Динамический тип

```
dynamic GetDate(DateFormats format)
{
    switch (format)
    {
        case DateFormats.ShortDateString:
            return DateTime.Now.ToShortDateString();
        case DateFormats.ShortTimeString:
            return DateTime.Now.ToShortTimeString();
        case DateFormats.Ticks:
            return DateTime.Now.Ticks;
        default:
            return DateTime.Now;
    }
}
```

Динамический тип

Пространство имен **System.Dynamic** предоставляет класс **ExpandoObject**, позволяющий создавать динамические объекты, наподобие тех, что используются в javascript:

Динамический тип

```
dynamic exObject = new ExpandoObject();
exObject.Name = "Bob";
exObject.GetName = (Func<int>)(() => exObject.Name);

Console.WriteLine(exObject.GetName());
```

Динамический тип

Использование класса **DynamicObject** пространства имен System.Dynamic – см.

https://docs.microsoft.com/en-us/dotnet/api/system.dynamic.dynamicobject?view=net-5.0

Сборки и метаданные

АТРИБУТЫ

Атрибуты

Атрибуты позволяют определить дополнительную информацию в метаданных, связанных с типом, и дают механизм для обращения к этой информации в ходе компиляции или выполнения программы.

Атрибуты

- Атрибуты добавляют в программу метаданные. Можно добавить пользовательские атрибуты, чтобы указать любую дополнительную информацию.
- Можно применить один или несколько атрибутов ко всей сборке, к модулю или к более мелким элементам программы, например к классам и свойствам.
- Атрибуты могут принимать аргументы, так же как методы и свойства.
- Программа может проверить собственные метаданные или метаданные в других программах, используя отражение.

Атрибуты

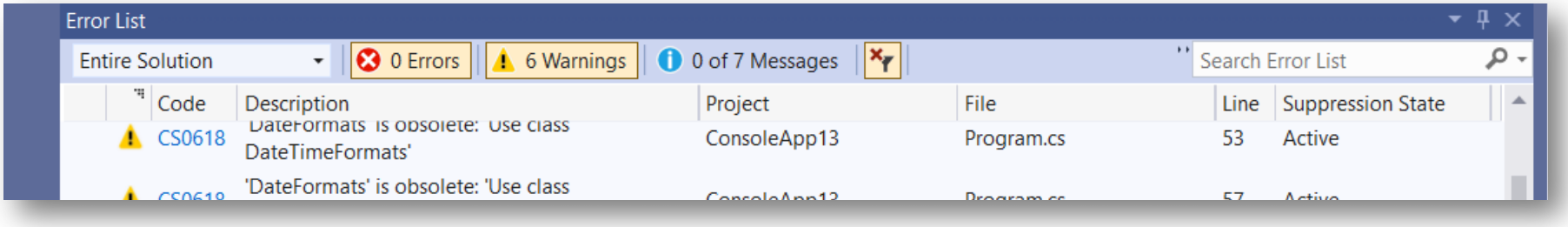
Чтобы указать атрибут на C#, поместите имя атрибута в квадратных скобках ([]) над объявлением сущности, к которой он применяется.

Атрибуты

[DllImport] - позволяет коду .NET отправлять вызовы в любую неуправляемую библиотеку кода на C или C++, в том числе и API-интерфейс лежащей в основе операционной системы.
[Obsolete] - позволяет указать, что данный тип или член является устаревшим. Когда другие программисты попытаются использовать элемент с таким атрибутом, они получат соответствующее предупреждение от компилятора
[Serializable] - позволяет указать, что класс или структура является "сериализируемой", т.е. способна сохранять свое текущее состояние в потоке
[NonSerialized] - позволяет указать, что данное поле в классе или структуре не должно сохраняться в процессе сериализации

Атрибуты

```
[Obsolete("Use class DateTimeFormats")]
enum DateFormats
{
    ShortDateString,
    ShortTimeString,
    Ticks
}
```



Атрибуты

```
public class Book
    {
        . . .

        [Conditional("Demo")]
        public void DemoMethod()
        { Console.WriteLine("DemoMeth"); }

        [Conditional("Release")]
        public void ReleaseMethod()
        { Console.WriteLine("ReleaseMeth"); }
```

Атрибуты

```
#define Demo
using System;

var book = new Book();
book.ReleaseMethod();
book.DemoMethod();
```

Атрибуты (пользовательские атрибуты)

Создадим атрибут, устанавливающий ограничения на длину строкового свойства класса

Атрибуты (пользовательские атрибуты)

```
[AttributeUsage(AttributeTargets.Property)]
class LengthConstraintAttribute : System.Attribute
{
    public int MinLength { get; set; } = 0;
    public int MaxLength { get; set; } = Int32.MaxValue;
    public LengthConstraintAttribute()
        {  }
    public LengthConstraintAttribute(int min)
        { MinLength = min; }
    public LengthConstraintAttribute(int min, int max)
        => (MinLength, MaxLength) = (min, max);
}
```

Атрибуты (пользовательские атрибуты)

```
class RegisterModel
    {
        [LengthConstraint(3,6)]
        public string Login { get; set; }
        [LengthConstraint(6)]
        public string Password { get; set; }
    }
```

Атрибуты (пользовательские атрибуты)

```
bool CheckStrings<T>(T data)
{
    var type = typeof(T);
    PropertyInfo[] props = type.GetProperties();
    foreach(var property in props)
    {
        object[] attrs = property.GetCustomAttributes(false);
        var result = true;
        foreach(LengthConstraintAttribute attr in attrs)
        {
            var length = ((string)property.GetValue(data)).Length;
            result &= attr.MinLength <= length
                      && attr.MaxLength >= length;
        }
        if (!result) return false;
    }
    return true;
}
```

Атрибуты (пользовательские атрибуты)

```
var model = new RegisterModel
                {Login="user", Password="12345" };
var isValid = CheckStrings(model);
```

Сборки и метаданные

ПРОЦЕССЫ И ДОМЕНЫ

Процессы и домены

При запуске приложения операционная система создает для него отдельный процесс, которому выделяется определённое адресное пространство в памяти

Процессы и домены

Процессы изолированы друг от друга, чтобы код, выполняемый одним приложением, не сможет нарушить работу других, не связанных с ним приложений.

Это значит, один процесс не имеет доступа в адресное пространство другого процесса, даже если оба процесса – результат запуска одного и того же приложения

Процессы и домены

Пространство имен **System.Diagnostics** предоставляет класс **Process**.

Этот класс позволяет управлять уже запущенными процессами, а также запускать новые.

Процессы и домены

- Свойство **Handle**: возвращает дескриптор процесса
- Свойство **Id**: получает уникальный идентификатор процесса в рамках текущего сеанса ОС
- Свойство **MachineName**: возвращает имя компьютера, на котором запущен процесс
- Свойство **Modules**: получает доступ к коллекции ProcessModuleCollection, которая хранит набор модулей (файлов dll и exe), загруженных в рамках данного процесса
- Свойство **ProcessName**: возвращает имя процесса, которое нередко совпадает с именем приложения
- Свойство **StartTime**: возвращает время, когда процесс был запущен

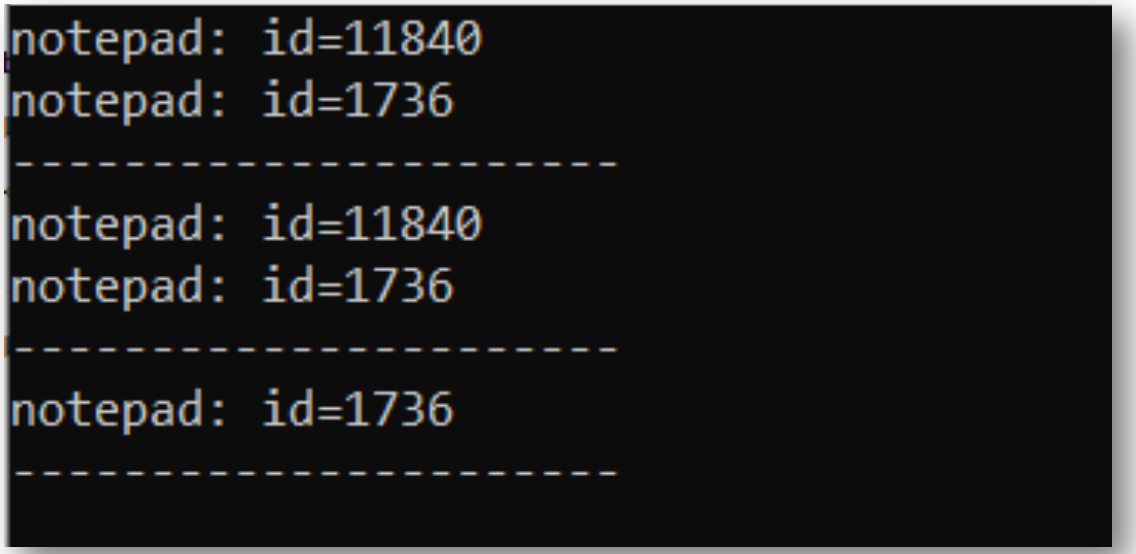
Процессы и домены

- Свойство **VirtualMemorySize64**: возвращает объем памяти, который выделен для данного процесса
- Метод **CloseMainWindow()**: закрывает окно процесса, который имеет графический интерфейс
- Метод **GetProcesses()**: возвращающий массив всех запущенных процессов
- Метод **GetProcessesByName()**: возвращает процессы по его имени. Так как можно запустить несколько копий одного приложения, то возвращает массив
- Метод **Kill()**: останавливает процесс
- Метод **Start()**: запускает новый процесс

Процессы и домены

```
var npad1 = Process.Start("Notepad.exe");
var npad2 = Process.Start("Notepad.exe","temp.txt");
foreach (var proc in Process.GetProcessesByName("notepad"))
    Console.WriteLine($"notepad: id={proc.Id}");
Console.WriteLine("----------------------");
npad1.Kill();
foreach (var proc in Process.GetProcessesByName("notepad") )
    Console.WriteLine($"notepad: id={proc.Id}");
Console.WriteLine("----------------------");
GC.Collect();
foreach (var proc in Process.GetProcessesByName("notepad"))
    Console.WriteLine($"notepad: id={proc.Id}");
Console.WriteLine("----------------------");
```

Процессы и домены



Процессы и домены

```
var pInfo = new ProcessStartInfo();
pInfo.FileName = "Notepad.exe";
pInfo.Arguments = "temp.txt";
var nPad = Process.Start(pInfo);
```

Процессы и домены (.Net Framework)

Домены приложений предоставляют среде CLR более безопасные и гибкие блоки, которые могут использоваться для разделения отдельных приложений. В одном процессе можно запустить несколько доменов приложений с таким же уровнем изоляции, какой обеспечивают отдельные процессы, но без дополнительных издержек на межпроцессные вызовы или переключение между процессами.

Процессы и домены (.Net Framework)

При запуске приложения, написанного на C#, операционная система создает процесс, а среда CLR создает внутри этого процесса логический контейнер, который называется **доменом приложения** и внутри которого работает запущенное приложение.

Процессы и домены (.Net Core, .Net5 и выше)

Для доменов требуется поддержка **среды выполнения**, при этом они требовательны к ресурсам.

Создание дополнительных доменов приложений **не поддерживается**, и в будущем эта возможность не планируется.

Для изоляции кода используйте в качестве альтернативы отдельные процессы или контейнеры.

Процессы и домены (.Net Core, .Net5 и выше)

Для динамической загрузки сборок используйте класс **AssemblyLoadContext** из пространства имен **System.Runtime.Loader**.

Процессы и домены (.Net Core, .Net5 и выше)

Для динамической загрузки сборок используйте класс **AssemblyLoadContext**.

Процессы и домены (.Net Core, .Net5 и выше)

Примечание

Методы Assembly.LoadFile(), Assembly.LoadFrom(), Assembly.Load() при загрузке библиотеки также используют AssemblyLoadContext (создают новый, либо либо используют AssemblyLoadContext.Default)

Процессы и домены (.Net Core, .Net5 и выше)

```
public class CustomAssemblyLoadContext : AssemblyLoadContext
{
    public CustomAssemblyLoadContext() : base(isCollectible: true)
    { }

    protected override Assembly Load(AssemblyName assemblyName)
    {
        return null;
    }
}
```

Процессы и домены (.Net Core, .Net5 и выше)

```
var context = new CustomAssemblyLoadContext();
// получаем путь к сборке MyApp
var   assemblyPath   =   Path.Combine(Directory.GetCurrentDirectory(),
"MyApp.dll");
// загружаем сборку
Assembly assembly = context.LoadFromAssemblyPath(assemblyPath);
// получаем тип MyClass из сборки MyApp.dll
var type = assembly.GetType("MyApp.MyClass");
// получаем его метод SomeMeth
var someMethod = type.GetMethod("SomeMeth");
//создаем экземпляр класса MyClass
var instance = Activator.CreateInstance(type);
// вызываем метод
int result = (int)someMethod.Invoke(instance, new object[] { 2 });
```

Процессы и домены (.Net Core, .Net5 и выше)

```
context.Unload();
```